

学号： 1906410229

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 动态规划(二)

2022年秋季学期

上机时间：第 7 周 星期 五 第 3 ~ 4 节

学生姓名： 蔡艺洵 专业班级： 计算机 1902

指导教师： 刘俊岭 成 绩：

项 目 名 称	动态规划(二)	学 时	4
日期： 2022 年 10 月 14 日		第 7 教学周	
硬件及软件			
名 称		版本、型号	
Dev-C++		5.11	
上机目的与要求：			
1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。			
本次任务：			
实验 3-3： 01 背包问题实验 实验 3-4： 完全背包 实验 3-5： 最长公共子序列 实验 3-6： 最大子段和			

实验 3-3 01 背包问题实验

1.问题描述:

有 n 个物品，每个物品有重量 `weight` 和价值 `profit` 两个属性，还有一个承重为 c 的背包。现在让你用这个背包装物品，使所装物品的价值最大化。

2.算法实例:

输入:

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

输出:

要求:

- 1 完成填空;
- 2 打印 `dp` 数组;
- 3 打印追踪数组 `rec[]`。

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-3.exe

```
放入背包的物品为:
第2个, 重量为: 2, 价值为: 6
第4个, 重量为: 5, 价值为: 16
总重量为: 7, 总价值为: 22
dp数组(二维)为:
dp[1][0]:0 dp[1][1]:1 dp[1][2]:1 dp[1][3]:1 dp[1][4]:1 dp[1][5]:1 dp[1][6]:1 dp[1][7]:1
dp[2][0]:0 dp[2][1]:1 dp[2][2]:6 dp[2][3]:7 dp[2][4]:7 dp[2][5]:7 dp[2][6]:7 dp[2][7]:7
dp[3][0]:0 dp[3][1]:1 dp[3][2]:6 dp[3][3]:10 dp[3][4]:11 dp[3][5]:16 dp[3][6]:17 dp[3][7]:17
dp[4][0]:0 dp[4][1]:1 dp[4][2]:6 dp[4][3]:10 dp[4][4]:11 dp[4][5]:16 dp[4][6]:17 dp[4][7]:22
rec数组(一维)为:
rec[1]:0 rec[2]:1 rec[3]:0 rec[4]:1 请按任意键继续. . .
```

3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void Knapsack(int profit[], int weight[], int c, int n, int **dp);
void Traceback(int **dp, int weight[], int c, int n, int *x);
```

```

int main(){
    int profit[5] = { 0, 1, 6, 10, 16 };
    int weight[5] = { 0, 1, 2, 3, 5 };
    int c = 7, n = 4;

    int ** dp = new int*[n + 1]; //共有 length1+1 行
    for (int i = 0; i < n + 1; i++)
        dp[i] = new int[c + 1]; //共有 length2+1 列
    int *x=new int[c + 1];

    int sumWeight=0, sumProfit=0;

    Knapsack(profit, weight, c, n, dp);
    Traceback(dp, weight, c, n, x);
    printf("放入背包的物品为: \n");
    for (int i = 1; i < c ; i++)
    {

        if (x[i] == 1){
            printf("第%d 个, 重量为: %d, 价值为: %d\n", i, weight[i], profit[i]);
            sumWeight += weight[i];
            sumProfit += profit[i];
        }
        //printf("%d ", x[i]);
    }
    printf("总重量为: %d, 总价值为: %d\n", sumWeight, sumProfit);

    for (int j = 0; j < weight[1]; j++)
    {
        dp[0][j] = 0;
    }
    for (int j = weight[1]; j <= c; j++)
    {
        dp[0][j] = profit[0];/**
    }
    for (int i = 1; i <= 4; i++)
    {
        for (int j = 0; j <= c; j++)
        {
            if(j<weight[i]) dp[i][j]=dp[i-1][j];
            else{
                dp[i][j]=max(dp[i - 1][j], dp[i - 1][j - weight[i]] + profit[i]);
            }
        }
    }
    }
    printf("dp 数组(二维)为: \n");

```

```

    for(int i=1;i<=n;i++){
        for(int j=0;j<=c;j++){
            printf("dp[%d][%d]:%d  ",i,j,dp[i][j]);
        }
        printf("\n");
    }
    printf("rec 数组(一维)为: \n");
    for(int i=1;i<=n;i++){
        printf("rec[%d]:%d  ",i,x[i]);
    }
    system("PAUSE");
    return 0;
}

void Knapsack(int profit[], int weight[], int c, int n, int **dp)
{
    int jMax = min(weight[n] - 1, c); //背包剩余容量上限 范围[0~w[n]-1]
    for (int j = 0; j <= jMax; j++)
    {
        dp[n][j] = 0;
    }

    for (int j = weight[n]; j <= c; j++) //限制范围[w[n]~c]
    {
        dp[n][j] = profit[n]; //**
    }

    for (int i = n - 1; i > 1; i--)
    {
        jMax = min(weight[i] - 1, c);
        for (int j = 0; j <= jMax; j++) //背包不同剩余容量 j<=jMax<c
        {
            dp[i][j] = dp[i + 1][j]; //没产生任何效益
        }

        for (int j = weight[i]; j <= c; j++) //背包不同剩余容量 j-wi >c
        {
            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j - weight[i]] + profit[i]); //效益值增长 vi
        }
    }
    dp[1][c] = dp[2][c];
    if (c >= weight[1])
    {
        dp[1][c] = max(dp[1][c], dp[2][c - weight[1]] + profit[1]);
    }
}

```

//x[]数组存储对应物品 0-1 向量,0 不装入背包, 1 表示装入背包

void Traceback(int **dp, int weight[], int c, int n, int *x)

```
{
    for (int i = 1; i < n; i++)
    {
        if (dp[i][c] == dp[i + 1][c])
        {
            x[i] = 0;
        }
        else
        {
            x[i] = 1;
            c -= weight[i];
        }
    }
    x[n] = (dp[n][c]) ? 1 : 0;
}
```

}}4.程序验证:

```
int profit[5] = { 0, 1, 6, 10, 16 };
int weight[5] = { 0, 1, 2, 3, 5 };
int c = 7, n = 4;
```

设定的 n 为 4, 背包容量 c 为 7, 重量数组为{ 0, 1, 2, 3, 5 }, 下标 0 的数据无效, 价值数组为{ 0, 1, 6, 10, 16 }, 下标 0 的数据无效, 结果如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-3.exe

放入背包的物品为:
第2个, 重量为: 2, 价值为: 6
第4个, 重量为: 5, 价值为: 16
总重量为: 7, 总价值为: 22

打印 dp 数组 (二维, 未优化时) 和追踪数组 rec 如下图所示:

```
dp数组(二维)为:
dp[1][0]:0 dp[1][1]:1 dp[1][2]:1 dp[1][3]:1 dp[1][4]:1 dp[1][5]:1 dp[1][6]:1 dp[1][7]:1
dp[2][0]:0 dp[2][1]:1 dp[2][2]:6 dp[2][3]:7 dp[2][4]:7 dp[2][5]:7 dp[2][6]:7 dp[2][7]:7
dp[3][0]:0 dp[3][1]:1 dp[3][2]:6 dp[3][3]:10 dp[3][4]:11 dp[3][5]:16 dp[3][6]:17 dp[3][7]:17
dp[4][0]:0 dp[4][1]:1 dp[4][2]:6 dp[4][3]:10 dp[4][4]:11 dp[4][5]:16 dp[4][6]:17 dp[4][7]:22
rec数组(一维)为:
rec[1]:0 rec[2]:1 rec[3]:0 rec[4]:1 请按任意键继续. . .
```

5.算法分析:

该算法使用了动态规划, 是动态规划类算法。因为要求打印 dp 数组, 为了更直观, 后面我将 dp 数组还原为了二维, 故此, 空间复杂度为 $n*c$ 。如果是设置为滚动数组, 空间复杂度为 n , 代码同样实现了。

时间复杂度: $O(n*c)$

空间复杂度: $O(n*c)$

实验 3-4 完全背包

1.问题描述:

有 n 件物品和一个最多能背重量为 c 的背包。第 i 件物品的重量是 $\text{weight}[i]$ ，得到的价值是 $\text{profit}[i]$ 。每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。

2.算法实例:

输入:

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

输出:

要求:

- 1 填空完成程序;
- 2 打印 dp 数组;
- 3 打印追踪数组 $\text{rec}[]$ 。

3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void completeKnapsack();

int max(int a,int b){
    if(a>b)
        return a;
    else
        return b;
}

int main() {
    completeKnapsack();
}
```

```

    system("PAUSE");
    return 0;
}

void completeKnapsack()
{
    vector<int> weight;
    vector<int> profit;
    weight.push_back(1);weight.push_back(2);weight.push_back(3);weight.push_back(5);
    profit.push_back(1);profit.push_back(6);profit.push_back(10);profit.push_back(16);
    int c = 7;//背包容量
    vector<int> dp(c + 1, 0);
    for (int i = 0; i < weight.size(); i++) { // 遍历物品
        cout << "i=" << i << " ";
        for (int j = 1; j <= c; j++) { // 遍历背包容量
            if (j < weight[i])
                dp[j] = dp[j];
            else
                dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
            cout << "j=" << j << "dp[j]=" << dp[j] << " ";
        }
        cout << endl;
    }
    cout << dp[c] << endl;
}

```

4.程序验证:

```

vector<int> weight;
vector<int> profit;
weight.push_back(1);weight.push_back(2);weight.push_back(3);weight.push_back(5);
profit.push_back(1);profit.push_back(6);profit.push_back(10);profit.push_back(16);
int c = 7;//背包容量

```

通过容器 vector 来记录重量 weight 和价值 profit，对 weight 容器依次压入数字 1，2，3，5。对容器 profit 依次压入数字 1，6，10，16。设置背包容量 c 为 7。结果如图所示：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-4.exe

```

i=0 j=1dp[j]=1 j=2dp[j]=2 j=3dp[j]=3 j=4dp[j]=4 j=5dp[j]=5 j=6dp[j]=6 j=7dp[j]=7
i=1 j=1dp[j]=1 j=2dp[j]=6 j=3dp[j]=7 j=4dp[j]=12 j=5dp[j]=13 j=6dp[j]=18 j=7dp[j]=19
i=2 j=1dp[j]=1 j=2dp[j]=6 j=3dp[j]=10 j=4dp[j]=12 j=5dp[j]=16 j=6dp[j]=20 j=7dp[j]=22
i=3 j=1dp[j]=1 j=2dp[j]=6 j=3dp[j]=10 j=4dp[j]=12 j=5dp[j]=16 j=6dp[j]=20 j=7dp[j]=22
22
请按任意键继续. . .

```

5.算法分析:

- 时间复杂度: $O(n \times c)$
- 空间复杂度: $O(n)$

实验 3-5 最长公共子序列

1.问题描述:

给定两个序列 X 和 Y，求两个序列最长公共子序列。

2.算法实例:

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

输入: X: "ABCB DAB" Y: "BDCABA"

输出:

db 数组:

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	1	1	1	1	2	2
3	0	1	1	2	2	2	2
4	0	1	1	2	2	3	3
5	0	1	2	2	2	3	3
6	0	1	2	2	3	3	4
7	0	1	2	2	3	4	4

rec 数组

	j=1	2	3	4	5	6
i=1	U	U	U	LU	L	LU
2	LU	L	L	U	LU	L
3	U	U	LU	L	U	U
4	LU	U	U	U	LU	L
5	U	LU	U	U	U	U
6	U	U	U	LU	U	LU
7	LU	U	U	U	LU	U

3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstring>

using namespace std;

int LCSLength(char* str1, char* str2, int** rec);
void PrintLCS(int** rec, char* str1, int i, int j);

int main()
{
    //双指针的方法申请动态二维数组
    char str1[50]="ABCBADAB";
    char str2[50]="BDCABA";
    int length1 = strlen(str1);
    int length2 = strlen(str2);

    int** rec = new int*[length1 + 1]; //共有 length1+1 行
    for (int i = 0; i < length1 + 1; i++)
        rec[i] = new int[length2 + 1]; //共有 length2+1 列

    printf("公共最长子序列长度为: %d\n", LCSLength(str1, str2, rec));
    printf("追踪为: \n");

    PrintLCS(rec, str1, length1, length2);

    printf("\n");
    for (int i = 0; i < length1 + 1; i++) //释放动态申请的二维数组
        delete[] rec[i];
    delete[] rec;
    system("PAUSE");
    return 0;
}

int LCSLength(char* str1, char* str2, int** rec)
{
    int i, j, length1, length2, len;
    length1 = strlen(str1);
    length2 = strlen(str2);

    //双指针的方法申请动态二维数组
```

```

int** c = new int*[length1 + 1]; //共有 length1+1 行
for (i = 0; i < length1 + 1; i++)
    c[i] = new int[length2 + 1]; //共有 length2+1 列
for (i = 0; i < length1 + 1; i++)
    c[i][0] = 0;           //第 0 列都初始化为 0
for (j = 0; j < length2 + 1; j++)
    c[0][j] = 0;           //第 0 行都初始化为 0
for (i = 1; i < length1 + 1; i++)
{
    for (j = 1; j < length2 + 1; j++)
    {
        if (str1[i - 1] == str2[j - 1]) //由于 c[i][0] 的 0 行 0 列没有使用, c[i][0] 的第 i 行元素对应 str1 的第 i-1 个
元素
        {
            c[i][j] = c[i - 1][j - 1] + 1;
            rec[i][j] = 0; //搜索方向为左上 LU
        }
        else if (c[i - 1][j] > c[i][j - 1])
        {
            c[i][j] = c[i - 1][j];
            rec[i][j] = 1; //搜索方向为上 U
        }
        else
        {
            c[i][j] = c[i][j - 1];
            rec[i][j] = -1; //搜索方向为左 L
        }
    }
}
for (i = 0; i < length1 + 1; i++)
{
    for (j = 0; j < length2 + 1; j++)
        printf("%d ", c[i][j]);
    printf("\n");
}

for (i = 1; i < length1 + 1; i++)
{
    for (j = 1; j < length2 + 1; j++)
        printf("%d ", rec[i][j]);
    printf("\n");
}

len = c[length1][length2];
for (i = 0; i < length1 + 1; i++) //释放动态申请的二维数组
    delete[] c[i];
delete[] c;

```

```

        return len;
    }
void PrintLCS(int** rec, char* str1, int i, int j)
{
    if (i == 0 || j == 0)
        return;
    if (rec[i][j] == 0)
    {
        PrintLCS(rec, str1, i - 1, j - 1); //从后面开始递归，所以要先递归到子串的前面，然后从前往后开始输出子串
        printf("%c", str1[i - 1]); //c[i][j]的第 i 行元素对应 str1 的第 i-1 个元素
    }
    else if (rec[i][j] == 1)
        PrintLCS(rec, str1, i - 1, j);
    else
        PrintLCS(rec, str1, i, j - 1);
}
}

```

4.程序验证:

```

char str1[50]="ABCADBCA";
char str2[50]="BACDBDA";
int length1 = strlen(str1);
int length2 = strlen(str2);

```

序列分别为"ABCADBCA"与"BACDBDA",验证其最长公共子序列,输入:X:"ABCADBCA" Y:"BACDBDA"
输出结果为 BADBA 结果如图所示:

```

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-5.exe
0 0 0 0 0 0 0 0
0 0 1 1 1 1 1 1
0 1 1 1 1 2 2 2
0 1 1 2 2 2 2 2
0 1 2 2 2 2 2 3
0 1 2 2 3 3 3 3
0 1 2 2 3 4 4 4
0 1 2 3 3 4 4 4
0 1 2 3 3 4 4 5
-1 0 -1 -1 -1 -1 0
0 -1 -1 -1 0 -1 -1
1 -1 0 -1 -1 -1 -1
1 0 -1 -1 -1 -1 0
1 1 -1 0 -1 0 -1
0 1 -1 1 0 -1 -1
1 1 0 -1 1 -1 -1
1 0 1 -1 1 -1 0
公共最长子序列长度为: 5
追踪为:
BADBA
请按任意键继续. . .

```

5.算法分析:

时间复杂度为 $O(m*n)$

空间复杂度为 $O(m*n)$

实验 3-6 最大子段和

1.问题描述:

给定 n 个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$, 找到一个具有最大和的连续子数组 (子数组最少包含一个元素), 返回其最大和。

2.算法实例:

$$\max\{0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k\}$$

输入: [4,-1,3,-2,5,-3,2,-1,1] 输出: 9

解释: 连续子数组 [4,-1,3,-2,5] 的和最大, 为 9

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-6.exe

最大和为: 9

请按任意键继续. . .

3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        if (nums.size() == 0) return 0;
        vector<int> dp(nums.size());
        vector<int> rec(nums.size());
        dp[0] = nums[0];
        rec[0] = 0;
        int result = 0;
        for (int i = 1; i < nums.size(); i++) {
            if (dp[i - 1] + nums[i] > nums[i]) {
                dp[i] = dp[i - 1] + nums[i];
                rec[i] = rec[i - 1];
            }
            else {
                dp[i] = nums[i];
                rec[i] = i;
            }
        }
    }
};
```

```

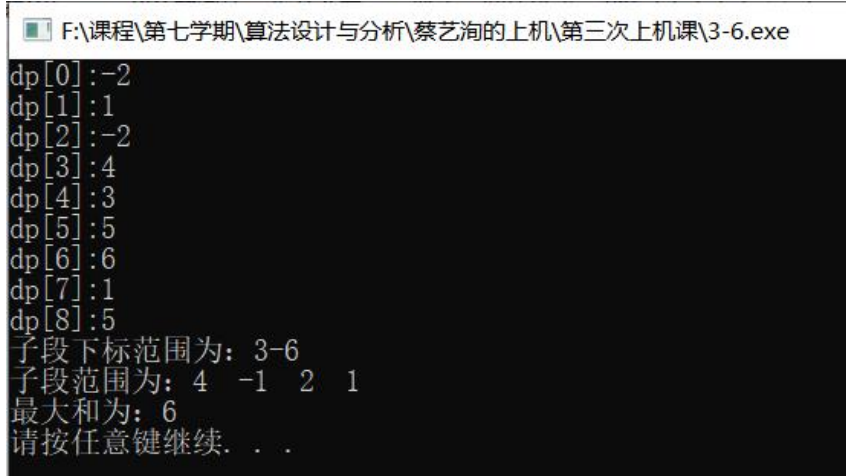
        //dp[i] = max(dp[i - 1] + nums[i], nums[i]); // 状态转移公式
        if (dp[i] > dp[result]) result = i; // result 保存 dp[i]的最大值的下标
    }
    for(int i=0;i<nums.size();i++){
        printf("dp[%d]:%d\n",i,dp[i]);
    }
    printf("子段下标范围为: %d-%d\n",rec[result],result);
    printf("子段范围为: ");
    for(int i=rec[result];i<=result;i++){
        printf("%d  ",nums[i]);
    }
    printf("\n");
    return dp[result];
}
};

int main() {
    int ans;
    Solution solution;
    vector<int> a;
    a.push_back(-2);a.push_back(1);a.push_back(-3);a.push_back(4);a.push_back(-1);
    a.push_back(2);a.push_back(1);a.push_back(-5);a.push_back(4);
    ans=solution.maxSubArray(a);
    cout << "最大和为: ";
    cout << ans << endl;
    system("PAUSE");
    return 0;
}

```

4.程序验证:

输入的序列为 [-2,1,-3,4,-1,2,1,-5,4],输出的 dp 数组为: -2, 1, -2, 4, 3, 5, 6, 1, 5。字段下标范围为 3-6, 字段范围为[4,-1,2,1], 最大和为: 6 结果如图所示:



```

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第三次上机课\3-6.exe
dp[0]:-2
dp[1]:1
dp[2]:-2
dp[3]:4
dp[4]:3
dp[5]:5
dp[6]:6
dp[7]:1
dp[8]:5
子段下标范围为: 3-6
子段范围为: 4 -1 2 1
最大和为: 6
请按任意键继续. . .

```

5.算法分析:

时间复杂度: $O(n)$

空间复杂度: $O(n)$

分析与总结：

动态规划类算法算是比较常用的方法，在力扣中很常见，与递归、分治、贪心、回溯等都是比较常见的，我本身也经常在力扣上刷题，目前算来，也快一年了，目前在上面都是比较偏向数据结构类的题目，例如数组、字符串、队列、堆栈、链表等，但是在刷这些题目时依旧会很常见的遇到动态规划类的解法，它和数据结构互为表里，相互辅助。动态规划在力扣上面有模板。

一般步骤为：

- 问题结构分析

给出问题表示，明确原始问题

- 递推关系建立

分析最优（子）结构，构造递推公式

- 自底向上计算

确定计算顺序，依次求解问题

- 最优方案追踪

记录决策过程，输出最优方案

本次上机 成绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：